

Quiz 3

Chapter 5: CPU Scheduling

1. Multiple-Processor Scheduling

- Scheduling becomes more complex when multiple CPUs are involved, with additional challenges in managing tasks efficiently across several processors.
- There are two types of multiple-processor scheduling:
 - **Asymmetric Multiprocessing**: Only one processor accesses system data structures, reducing the need for data sharing and simplifying data synchronization. This approach simplifies system design but does not fully utilize the capabilities of multiple processors.
 - **Symmetric Multiprocessing (SMP)**: Each processor in an SMP system is self-scheduling and independently decides which process to execute next. SMP systems may use a common ready queue shared by all processors, or each processor may maintain its own private queue of ready processes. SMP improves parallelism and efficiency, making it the preferred approach in modern systems.

2. Load Balancing in SMP

- **Load Balancing** ensures all CPUs work efficiently without any being idle while others are overloaded.
- Two main approaches to load balancing are used in SMP systems:
 - **Push Migration**: A periodic task checks the load of each processor. When an imbalance is detected, tasks are pushed from overloaded processors to those with lighter workloads, preventing bottlenecks and improving resource utilization.
 - **Pull Migration**: Idle processors actively seek tasks from busier processors. When a processor becomes idle, it pulls tasks from other busy processors, ensuring all processors remain effectively engaged.

3. Priority-Based Scheduling

- Real-time scheduling requires **preemptive, priority-based scheduling** to ensure that higher-priority processes get CPU time when needed.
- Soft real-time systems can work well with priority-based scheduling, but **hard real-time systems** require guarantees to meet strict deadlines.
- **Periodic Processes** have specific characteristics:
 - Each process has a defined **Processing Time (t)**, **Deadline (d)**, and **Period (p)**, where $t \leq d \leq p$. This ensures the process completes within its time frame.
 - The **Rate of a Periodic Task** is defined as $(1/p)$, representing how often the task must complete within its period.

4. Rate Monotonic Scheduling (RMS)

- **Priority Assignment** in RMS is based on the inverse of the task period. Tasks with shorter periods are assigned higher priority, while tasks with longer periods receive lower priority.

- This approach is optimal for fixed-priority scheduling with periodic tasks. Assigning shorter periods higher priority ensures tasks execute frequently enough to meet their constraints.
- **Example:** Given processes P1, P2, and P3 with different capacities and periods, RMS assigns priorities based on period length. Shorter period tasks, such as P2 with a period of 5, are given higher priority compared to tasks like P1 with a period of 20.

5. Earliest Deadline First (EDF) Scheduling

- In **EDF Scheduling**, priorities are assigned based on deadlines rather than fixed periods.
- Tasks with earlier deadlines are given higher priority, ensuring the system completes tasks that must be finished soonest.
- **Example:** Processes P1, P2, and P3 are prioritized according to their respective deadlines. The task with the nearest deadline, such as P2 with a deadline of 4, is given the highest priority to ensure it completes on time.

Chapter 6: Synchronization Tools

1. Background on Synchronization

- **Cooperating Processes** are processes that can impact or be impacted by the actions of other processes in the system. They often need to share resources or data to operate effectively.
- **Data Sharing** between cooperating processes can occur in two ways:
 - Directly, by sharing a **logical address space**, which includes both code and data.
 - Indirectly, through **files or messages**, which allows controlled communication and data sharing.
- **Concurrent access** to shared data can lead to **data inconsistency**, resulting in incorrect outcomes.
- **Process Synchronization** mechanisms are used to ensure orderly execution and maintain data consistency and correctness.

2. Producer-Consumer Problem

- The **Producer-Consumer Problem** illustrates the need for synchronization when multiple processes share a **buffer** in memory.
- Two types of buffers can be used:
 - **Unbounded Buffer:** No size limit. The producer can keep producing items indefinitely, while the consumer waits for available items. This ensures the producer is never blocked, but the consumer may need to wait.
 - **Bounded Buffer:** Fixed size. The buffer can only hold a limited number of items. The producer must wait if the buffer is full, and the consumer must wait if it is empty. This model ensures efficient resource use but requires careful management of producer and consumer activities.
- A **counter** variable keeps track of the number of full buffers, ensuring the producer and consumer remain synchronized when accessing and modifying the buffer.

3. Race Condition

- A **Race Condition** occurs when the outcome of executing processes depends on the order or timing of their execution. Race conditions lead to inconsistencies and unpredictable behavior if

not handled properly.

- **Example of a Race Condition:**

- Consider a scenario where both the producer and consumer update a shared **counter** variable. If the operations are interleaved, the final value of the counter may be incorrect.
- For instance, starting with **count = 5**, the producer might increment the counter, while the consumer decrements it concurrently. The following interleaving could occur:
 - **S0:** Producer executes `register1 = counter` (`register1 = 5`).
 - **S1:** Producer executes `register1 = register1 + 1` (`register1 = 6`).
 - **S2:** Consumer executes `register2 = counter` (`register2 = 5`).
 - **S3:** Consumer executes `register2 = register2 - 1` (`register2 = 4`).
 - **S4:** Producer writes back `counter = register1` (`counter = 6`).
 - **S5:** Consumer writes back `counter = register2` (`counter = 4`).
- As a result, the **final value** of the counter is **4** instead of the correct value **6**, showing the inconsistency caused by the race condition.
- Synchronization tools and mechanisms are needed to prevent race conditions and ensure shared resources are accessed consistently and predictably.

Chapter 6: Synchronization Tools

1. Critical Section Problem

- **Definition:** A critical section is a segment of code where a process accesses shared resources. Only one process should be in its critical section at a time to prevent conflicts and ensure data consistency.
- **Goal:** The goal is to design a synchronization protocol that ensures mutual exclusion, so no two processes can be in their critical sections simultaneously, preventing data corruption.
- **Structure:**
 - **Entry Section:** Code that requests permission to enter the critical section.
 - **Critical Section:** The section where shared resources are accessed. Mutual exclusion is essential here.
 - **Exit Section:** Code executed after the critical section to release the lock, allowing others to proceed.
 - **Remainder Section:** The rest of the code that does not involve shared resources.

2. Solution Requirements for Critical Section Problem

- **Mutual Exclusion:** Only one process can be in its critical section at any time, preventing race conditions and data inconsistency.
- **Progress:** If no process is in the critical section and others want to enter, one of them must be allowed to proceed without delay. This ensures system progress.
- **Bounded Waiting:** There must be a limit on the number of times other processes can enter their critical sections after a process has requested access, preventing indefinite waiting (starvation).

3. Peterson's Solution

- **Overview:** Peterson's solution is a software-based synchronization mechanism for two processes, ensuring mutual exclusion. It demonstrates core synchronization concepts effectively.
- **Variables:**
 - `int turn` : Indicates which process has the right to enter the critical section.
 - `boolean flag[2]` : Indicates if a process wants to enter the critical section (`true` means interested).
- **Algorithm:**
 - A process signals its intent by setting its flag to `true` and gives `turn` to the other process, showing willingness to wait.
 - A process enters the critical section only if the other is not interested or if it is its turn. This ensures fairness and mutual exclusion.
- **Properties:**
 - **Mutual Exclusion:** Only one process can enter the critical section at a time.
 - **Progress:** If one process wants to enter, it can do so without waiting unnecessarily.
 - **Bounded Waiting:** Each process gets a fair opportunity without indefinite delays.

4. Synchronization Hardware

- **Hardware Support:** Many systems offer hardware-level support for efficient critical section handling using atomic operations that prevent race conditions.
- **Locking Mechanisms:** Locks protect critical regions by ensuring only one process can enter at a time.
- **Uniprocessor Systems:** Disabling interrupts temporarily can ensure mutual exclusion, but this approach is impractical for multiprocessor systems.

5. Locks for Critical Section Solutions

- **Lock-Based Approach:** Locks are used to manage access to critical sections. They range from basic binary locks to more advanced reader-writer locks.
- **Algorithm:**
 - **Acquire Lock:** Before entering the critical section, a process must acquire a lock, ensuring exclusive access.
 - **Critical Section:** The process performs operations without interference.
 - **Release Lock:** The process releases the lock after completing its work.
- **Benefits:** Locks effectively provide mutual exclusion and are scalable for systems with many processes. They prevent race conditions by controlling access to shared resources.
- **Drawbacks:** Locks can cause **deadlock** (when processes wait indefinitely for each other to release locks) and **priority inversion** (when a high-priority process waits for a lower-priority one to release a lock). Proper design is needed to prevent these issues.

Chapter 6: Synchronization Tools

1. `test_and_set` Instruction

- **Definition:** A hardware instruction used for synchronization.
 - The `test_and_set` operation checks a value and sets it to `TRUE` in one atomic action.

- Used to implement locking mechanisms to prevent concurrent access to critical sections.
- **Algorithm Using `test_and_set`:**
 - A shared lock variable is initialized to `FALSE`.
 - A process waits if `test_and_set(&lock)` returns `TRUE`. Otherwise, it enters the critical section and releases the lock when done.

2. Semaphores

- **Definition:** A synchronization tool that provides a mechanism to control access to shared resources more effectively than simple locks.
- **Types of Semaphores:**
 - **Counting Semaphore:** Can take on any non-negative integer value. Used to manage access to a pool of resources.
 - **Binary Semaphore (Mutex):** Takes values 0 or 1. Often used as a mutual exclusion lock.
- **Operations:**
 - **`wait()`:** Decreases the semaphore value if it is greater than 0. Otherwise, the process waits.
 - **`signal()`:** Increases the semaphore value, potentially allowing a waiting process to proceed.

Chapter 7: Classical Synchronization Problems

1. Bounded-Buffer Problem (Producer-Consumer Problem)

- **Scenario:** A buffer with `n` slots is shared between two processes: a **producer** that inserts items and a **consumer** that removes them.
- **Challenges:**
 - The producer must not add items if the buffer is full.
 - The consumer must not remove items if the buffer is empty.
 - Producer and consumer must not access the buffer simultaneously.
- **Solution:**
 - Use `**semaphore** mutex` (binary semaphore initialized to 1) for mutual exclusion.
 - Use `**semaphore** empty` (initialized to `n`) to keep track of empty slots.
 - Use `**semaphore** full` (initialized to 0) to track filled slots.
 - **Producer Process:** Waits on `empty` and `mutex` before adding an item, signals `full` after.
 - **Consumer Process:** Waits on `full` and `mutex` before removing an item, signals `empty` after.

2. Readers-Writers Problem

- **Scenario:** Multiple **readers** and **writers** access a shared dataset.
 - **Readers** only read; **Writers** can both read and write.
- **Challenges:**
 - Multiple readers can read simultaneously, but only one writer can access the data at a time.
 - Writers must have exclusive access.

- **Solution:**
 - Use `**semaphore **wrt_mutex` (initialized to 1) to ensure exclusive access for writers.
 - Use `**semaphore **mutex` (initialized to 1) to manage reader count.
 - Use `read_count` (an integer variable initialized to 0) to track the number of readers.
 - **Writer Process:** Waits on `wrt_mutex`, performs writing, then signals `wrt_mutex`.
 - **Reader Process:** Waits on `mutex` to increment `read_count`. The first reader waits on `wrt_mutex` to ensure no writers are active. Signals `mutex` after incrementing. Decrements `read_count` after reading, signaling `wrt_mutex` if no readers remain.

3. Dining-Engineers Problem

- **Scenario:** Five engineers sit around a table, alternating between thinking and eating. Each needs two **chopsticks** to eat.
- **Challenges:**
 - If each philosopher picks up one chopstick and waits for the other, a **deadlock** may occur where no engineer can proceed.
- **Solution:**
 - Use `**semaphores **chopstick[5]` (each initialized to 1) to represent chopsticks.
 - A philosopher waits on the two chopsticks before eating and signals them after.
 - **Deadlock Prevention Strategies:**
 - Allow only four philosophers to sit at the table at a time.
 - A philosopher can pick up chopsticks only if both are available.
 - **Asymmetric Solution:** Odd philosophers pick up the left chopstick first, even philosophers pick up the right chopstick first.

Chapter 8: Deadlocks

1. System Model

- **Resources:** A system consists of various types of resources, such as **CPU cycles, memory, and I/O devices**. Each resource type may have multiple **instances**. For example, memory might have several blocks, or an I/O device may have multiple channels. Resources are typically categorized based on their functionality and how they can be allocated to different processes.
- **Process Resource Usage:** Each process must interact with resources in a specific sequence: **request, use, and release**. First, a process must request the necessary resource(s). If the resource is available, the process can proceed; otherwise, it must wait. Once the process has the resource, it can use it for a specified task. Finally, once the task is complete, the process must release the resource. A **deadlock** can arise if multiple processes hold some resources while waiting indefinitely for others, thus blocking further progress.

2. Deadlock Characterization

- **Conditions for Deadlock:** Deadlock can occur only if the following four conditions hold simultaneously. Understanding each condition is crucial to designing strategies for deadlock management.

1. **Mutual Exclusion:** At least one resource must be held in a **non-shareable mode**, meaning only one process at a time can use it. If another process requests that resource, it must wait until it is released. For example, printers and tape drives can only be used by one process at a time, which makes them prone to mutual exclusion.
2. **Hold and Wait:** A process holding at least one resource is **waiting** rce is **waiting to acquire additional resources** that are currently being held by **oto acquire additional resources** that are currently being held by o potential contention.
3. **No Preemption:** Resources cannot be forcibly taken away from a process holding them; they can only be released **voluntarily** by the process after it has completed its task. This makes it difficult to resolve deadlock because resources cannot be reassigned until the process is finished.
4. **Circular Wait:** There must exist a set of processes $\{P_0, P_1, \dots, P_n\}$ such that **P0 is waiting for a resource held by P1, P1 is waiting for a resource held by P2, ..., Pn is waiting for a resource held by P0**, forming a circular chain. This cyclic dependency means that none of the processes can proceed, causing a complete standstill.

3. Methods for Handling Deadlocks

- **Prevention and Avoidance:** The primary goal is to **prevent the system from entering a deadlock state** by negating one of the four necessary conditions.
 - **Deadlock Prevention:** This strategy ensures that at least one of the four conditions for deadlock cannot hold. Examples include:
 - Eliminating **hold and wait** by requiring processes to request all resources at once, which may lead to inefficient resource usage.
 - Allowing **preemption**, where a resource held by a waiting process can be forcibly taken away.
 - **Deadlock Avoidance:** This approach uses additional information about how resources will be requested. The system dynamically examines the resource-allocation state to ensure it will not enter an unsafe state that could lead to deadlock. **Banker's Algorithm** is a well-known example of this technique.
- **Detection and Recovery:** In this approach, the system **allows deadlocks to occur** but has a mechanism in place to detect and recover from them. The system must maintain information about the current allocation of resources and periodically check for circular wait conditions. If a deadlock is detected, recovery techniques such as **resource preemption** or **process termination** are employed.
- **Ignore the Problem:** Some systems, such as **UNIX**, simply ignore the deadlock problem, assuming that it occurs infrequently and that manual intervention can resolve issues when they arise. This approach is known as the **ostrich algorithm**, as it "buries its head in the sand" regarding deadlocks.

4. Deadlock Avoidance

- **Resource Allocation State:** Deadlock avoidance requires the system to maintain a thorough understanding of the **resource-allocation state**, which includes the number of available resources, the number of allocated resources, and the maximum demands of the processes.

This helps in making informed decisions about whether to grant a resource request or make the process wait.

- **Safe and Unsafe States:**

- **Safe State:** The system is in a safe state if there exists a sequence of all processes such that each process can obtain the needed resources, execute, release resources, and terminate without causing deadlock. In a safe state, the system can guarantee that every process will complete.
- **Unsafe State:** If the system cannot guarantee that all processes will complete, it is in an unsafe state, which may lead to deadlock. The goal of deadlock avoidance is to keep the system in a safe state at all times.

- **Banker's Algorithm:** This algorithm simulates resource allocation for each possible request before deciding whether to grant it. By doing this, it ensures that the system will always remain in a **safe state**. Each process must declare the maximum number of resources it might need, and the system only grants requests that will keep it in a safe state.

5. Avoidance Algorithms

- **Single Instance:** When there is only one instance of each resource type, a **resource allocation graph** can be used. In this graph, nodes represent processes and resources, and directed edges indicate current allocations or pending requests. If a cycle forms, deadlock is possible.
- **Multiple Instances:** When multiple instances of resources exist, **Banker's Algorithm** is used. This algorithm evaluates the resource allocation and determines if granting a request keeps the system safe.

6. Banker's Algorithm

- **Purpose:** The Banker's Algorithm is used to ensure that resource allocation will not lead the system into an unsafe state. It operates by making sure that there is a sequence of resource allocations in which all processes can complete.
- **Key Information Required:**
 - **Maximum Resources:** The maximum demand of each process for each resource type.
 - **Current Allocation:** How many instances of each resource are currently allocated to each process.
 - **Availability:** The number of resource instances currently available in the system.
 - **Need Matrix:** Represents the remaining resources each process may need to complete its task. It is calculated as $Need[i] = Max[i] - Allocation[i]$.
- **Safe Sequence:** A **safe sequence** is an ordering of processes such that each can get its required resources, run to completion, release its resources, and allow the next process in the sequence to proceed. Finding a safe sequence ensures that the system will not enter deadlock.

7. Deadlock States

- **Safe State:** The system is in a safe state if a sequence of processes exists that guarantees each process can finish executing without causing a deadlock. A safe state means that the system can always find a way to allocate resources such that all processes can complete successfully.

- **Unsafe State:** An unsafe state means that the system cannot guarantee that all processes will complete without deadlock. Entering an unsafe state does not necessarily mean deadlock will occur, but it means the potential exists.

Chapter 8: Deadlocks

8. Deadlock Avoidance Algorithms

- **Single Instance of a Resource Type:** Use a **resource-allocation graph** to track resource requests and allocations. If a cycle forms in this graph, a deadlock is possible.
- **Multiple Instances of a Resource Type:** Use the **Banker's Algorithm** to evaluate resource allocation requests and ensure the system does not enter an unsafe state.

9. Banker's Algorithm

- **Purpose:** The **Banker's Algorithm** is used to prevent deadlock in systems with multiple instances of resources by ensuring that each resource request keeps the system in a safe state.
- **Required Information:**
 - **Maximum Resources:** Maximum number of instances each process may need for each resource type.
 - **Current Allocation:** Number of resource instances currently allocated to each process.
 - **Availability:** Number of resource instances available in the system.
 - **Need Matrix:** Indicates the remaining resources each process needs to complete its task ($\text{Need}[i] = \text{Max}[i] - \text{Allocation}[i]$).
- **Steps:**
 - When a process requests resources, the algorithm checks if granting the request keeps the system in a safe state.
 - If it is safe, the request is granted; if not, the process must wait.

10. Resource-Allocation Graph

- **Definition:** The **resource-allocation graph** is a tool used to model the state of resource allocation in a system.
 - **Vertices (V):** Partitioned into **Processes (P)** and **Resources (R)**.
 - **Edges:** Represent requests and assignments.
 - **Request Edge:** Directed edge from process P_i to resource R_j , indicating that P_i is requesting R_j .
 - **Assignment Edge:** Directed edge from resource R_j to process P_i , indicating that R_j is allocated to P_i .
- **Cycle Detection:**
 - **No Cycle:** No deadlock is possible.
 - **Cycle Present:**
 - If there is only one instance of each resource type, a cycle indicates a **deadlock**.
 - If multiple instances exist, a cycle means a **possibility** of deadlock but not a certainty.

11. Deadlock Prevention Techniques

- **Mutual Exclusion:** Make all resources **sharable** where possible (e.g., read-only files). Non-sharable resources must maintain mutual exclusion.

- **Hold and Wait:** Ensure that processes either request all resources at once or request only when none are held. This prevents processes from holding some resources while waiting for others, which may lead to deadlock.
- **No Preemption:** If a process holding some resources requests additional resources that are not available, all resources currently held are released. This prevents indefinite holding, but is practical only for preemptable resources.
- **Circular Wait:** Impose a global ordering on resource types and require each process to request resources in an increasing order. This breaks the circular chain of dependency, preventing deadlock.

Chapter 8: Deadlocks

1. Deadlock Detection

- **Approach:** Unlike prevention and avoidance, **deadlock detection** allows the system to enter a deadlock state. Once a deadlock occurs, an algorithm is used to detect it, and a recovery scheme is initiated to handle it.
- **Detection Algorithm:** The system must run an algorithm periodically to identify deadlocks. Depending on the type of resources (single or multiple instances), different algorithms are used.

2. Single Instance of Each Resource Type

- **Wait-for Graph:** To detect deadlock in systems with a single instance of each resource, the **wait-for graph** is used.
 - **Nodes** represent processes.
 - **Edges** represent that a process P_i is waiting for another process P_j to release resources.
- **Cycle Detection:** The algorithm looks for cycles in the graph. If a cycle is found, it means a deadlock exists. The complexity of detecting a cycle in a graph is $O(n^2)$, where n is the number of vertices (processes).

3. Multiple Instances of a Resource Type

- **Data Structures:**
 - **Available:** A vector of length m that indicates the number of available resources of each type.
 - **Allocation:** An $n \times m$ matrix that indicates how many resources of each type are currently allocated to each process.
 - **Request:** An $n \times m$ matrix that indicates how many resources each process is currently requesting. If $Request[i][j] = k$, it means that process P_i is requesting k more instances of resource type R_j .
- **Banker's Algorithm:** This algorithm is also used for deadlock detection in systems with multiple instances of resources. It checks whether current and future requests can be safely granted without leading to deadlock.

4. Recovery from Deadlock

- **Process Termination:**

- **Abort All Deadlocked Processes:** This brute-force approach terminates all processes involved in the deadlock.
- **Abort One Process at a Time:** Processes are aborted one by one until the deadlock cycle is broken.
- **Criteria for Choosing a Process to Abort:**
 1. **Priority** of the process.
 2. **Runtime:** How long the process has been running and how much more time is required for completion.
 3. **Resources Used:** The number of resources the process has already used.
 4. **Resources Needed:** The resources the process needs to complete.
 5. **Number of Processes to Terminate:** Minimizing the number of terminated processes.
 6. **Interactivity:** Whether the process is interactive or batch.
- **Resource Preemption:**
 - **Victim Selection:** Resources can be preempted from certain processes to resolve deadlocks. The goal is to **minimize the cost** of preemption.
 - **Rollback:** After preemption, processes may need to be **rolled back** to an earlier safe state to ensure consistency.
 - **Starvation:** To prevent a process from being repeatedly selected as a victim, a **cost factor** is used that includes the number of times a process has been rolled back.

5. Key Differences Between Resource-Allocation Graph and Wait-for Graph

- **Resource-Allocation Graph (RAG):** Represents both processes and resources. It is used to determine potential deadlock conditions by mapping out resource requests and allocations.
- **Wait-for Graph:** Derived from the resource-allocation graph by removing resource nodes and combining edges to directly show which processes are waiting for others. This is used primarily for systems with single instances of resources to detect cycles more easily.